Full-Stack Web Platform Architecture & Learning Guide

Encrypted IoT Device Fingerprinting & Anti-Spoofing Demonstration System

1. Executive Overview & Educational Purpose

1.1 Why We Built a Full-Stack Web Application

In modern cybersecurity and machine learning research, algorithms and mathematical formulas often remain trapped inside terminal scripts and Jupyter notebooks. While terminal scripts prove statistical viability, they suffer from three major educational limitations:

1. **The "Black Box" Problem:** Evaluators and learners cannot see the continuous temporal flow of network packets. Seeing a terminal printout of `Accuracy: 0.9934` does not build an intuitive understanding of *why* an Amazon Echo differs from a Philips Hue bridge.
2. **Lack of Interactive Experimentation:** In a static paper, you cannot dynamically drag a confidence slider to see how raising the anomaly threshold from $\tau = 0.80$ to $\tau = 0.90$ affects false alarm rates on real devices.
3. **Inability to Test Custom Traffic:** A real security tool must allow an operator to paste arbitrary packet captures and receive an instant, transparent classification breakdown.

To solve these challenges, we built an authentic, decoupled **full-stack web platform** (`iot_fingerprint_gui/`). It transforms theoretical timing moments into live, interactive visual elements where every mathematical calculation is exposed and explained in real time.

1.2 Who This Guide Is For

- **Students & Learners:** To understand the end-to-end journey of an encrypted network packet through a feature-extraction and classification pipeline.
- **Evaluators & Examiners:** To inspect the live software architecture, verify that results are computed legitimately (not hard-coded), and test edge cases.
- **Network Engineers:** To see how passive timing analysis can be integrated into modern edge security gateways without breaking TLS encryption or requiring payload decryption.

---

2. High-Level System Architecture & Technology Stack

The platform is designed with a **decoupled client-server architecture**:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
FRONTEND: REACT 19 + VITE 8.3 ■ ■ (Running on Port 5173) ■ ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■ ■ ■ Tab 1: Known (5) ■ ■ Tab 2:
Unknown (22)■ ■ Tab 3: Test Own ■ ■ Tab 4: Spoof ■■ ■ ■ Replay Streaming ■ ■ Open-Set Slider ■ ■ Custom Ingestion
■ ■ Verification■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
HTTP REST (JSON) ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
BACKEND: FASTAPI + UVICORN ■ ■ (Running on Port 8765) ■ ■ ■ ■ /model/info ■ /predict/csv ■ /stream/{dev} ■
/spoof/verify ■ /health ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ INFERENCE & MODEL PIPELINE (Python) ■ ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■ ■ ■ Raw Timestamp Pipe ■■>■ 13 Moments Engine■■>■ StandardScaler
■■>■ 100-Tree RF ■ ■ ■ ■ Finite Checks ■ ■ log1p(IAT) ■ ■ (raw_stat) ■ ■ (rf_model.pkl) ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

2.1 Technology Stack Rationale

- **FastAPI Backend (Python 3.12, Uvicorn on Port 8765):**

* *Why:* Python is the industry standard for scientific computing and scikit-learn model serving. FastAPI provides asynchronous request handling, automatic OpenAPI/Swagger documentation (`/docs`), and rigorous Pydantic schema validation.

- **React 19 Frontend (Vite 8.3 on Port 5173):**

* *Why:* React's component-driven state architecture allows individual charts, probability bars, and tables to re-render independently at 60 FPS without reloading the entire page. Vite provides instant Hot Module Replacement (HMR).

- **Tailwind CSS:**

* *Why:* Clean, modern dark-mode interface designed for high readability during technical demonstrations and defense presentations.

- **Recharts & Lucide Icons:**

* *Why:* High-performance SVG charts for rolling packet timelines and responsive UI icons.

---

3. The 7-Step Data Journey (From Packet to Pixel)

When a packet stream is evaluated in the dashboard, it traverses 7 exact mathematical transformations:

```
[Packet Timestamps] ■■> [Delta-t (IAT)] ■■> [Log Transform] ■■> [13 Features] ■ [Decision & Display] <■■ [Rejection
Engine] <■■ [Soft Averaging] <■■ [Random Forest]
```

1. **Step 1: Timestamp Ingestion & Sanitization (`backend/pipeline.py`):**
- The client sends an array of timestamps (e.g., $[t_0, t_1, \dots, t_{128}]$).
- The backend validates that all values are finite (`np.isfinite`), immediately rejecting `NaN`, $+\infty$, or $-\infty$ with an HTTP 400 error.
- Timestamps are sorted chronologically: $t_0 \le t_1 \le t_2 \dots$.

2. **Step 2: Inter-Arrival Time (IAT) Differencing:**
- Computes consecutive time differences: $\Delta t_i = t_i - t_{i-1}$.
- Rejects clock inversions by clamping to zero: $\text{IAT}_i = \max(\Delta t_i, 0.0)$.

3. **Step 3: Logarithmic Scale Compression:**
- Applies the fixed non-parametric transformation: $z_i = \ln(1 + \text{IAT}_i)$.
- Compresses extreme multi-minute sleep intervals while keeping millisecond packet bursts distinct.

4. **Step 4: Statistical Moments Calculation (`src/features.py`):**
- Computes the 13 summary moments (mean, std, min, max, median, skewness, kurtosis, p25, p75, p95, p99, energy, entropy).

5. **Step 5: Training Z-Score Standardization:**
- Features are standardized using the frozen training scaler: $\mathbf{x}_{\text{norm}} = (\mathbf{x} - \boldsymbol{\mu}_{\text{train}}) / \boldsymbol{\sigma}_{\text{train}}$.

6. **Step 6: Random Forest Ensemble Inference:**
- The scaled 13-element vector is passed through 100 decision trees.
- The model outputs a probability vector across the 5 enrolled classes: $\mathbf{P} = [P_{\text{Echo}}, P_{\text{Belkin}}, P_{\text{Hue}}, P_{\text{Camera}}, P_{\text{Plug}}]$.

7. **Step 7: Calibrated Thresholding & UI Rendering:**
- If $\max(\mathbf{P}) \ge 0.84$, the top class is accepted as `KNOWN`.
- If $\max(\mathbf{P}) < 0.84$, it is flagged as `UNKNOWN / Unfamiliar Device`.
- The JSON response is transmitted via REST to React, updating the UI in $< 15 \text{ milliseconds}$.

---

4. Deep Dive into Every Screen & UI Component

4.1 Global Navigation Bar (`Navbar.jsx`)

The header persistent across all views contains:

- **Branding & System Status:** Displays "Encrypted IoT Device Fingerprinting" and an active backend connectivity pulse.
- **Calibrated Threshold Pill (`Threshold: 0.84`):** A persistent visual reminder of the mathematically derived confidence boundary ($\tau = 0.84$).
- **Dark / Light Mode Toggle:** Supports high-contrast viewing for sunny conference rooms or dark laboratory environments.
- **Four Main Navigation Tabs:**

1. `Known Devices (5)`
2. `Unknown Devices (22)`
3. `Test Your Own Device`
4. `Spoofing Detection`

4.2 Tab 1: Known Devices — Replay & Feature Analysis (`TabKnown.jsx`)

Educational Purpose

To demonstrate the **closed-set baseline performance** of the 5 authorized enrolled hardware devices and illustrate how distinct their timing rhythms are.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ DEVICE SELECTOR: [
Amazon Echo ] [ Belkin Motion ] [ Philips Hue ]... ■ ■ CONTROLS: [ Play ] [ Pause ] [ Step ] Speed: 1x ■■●■■ 5x ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ROLLING IAT TIMELINE
■ PROBABILITY BARS ■ ■ (Live line chart of last 128 ■ Amazon Echo: [■■■■■■■■■■] 100% ■ ■ packet gaps in seconds)
■ Philips Hue: [ ] 0% ■ ■ ■ Belkin Motion: [ ] 0% ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ 13 STATISTICAL
TIMING MOMENTS (Live Updating) ■ ■ Mean: 0.0412 | Std: 0.0521 | Median: 0.0210 | Min: 0.0001 ■ ■ P25: 0.0041 | P75:
0.0612 | Skew: 1.2410 | Entropy: 4.812 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Key UI Elements Explained

1. **Device Selector Buttons:** Switches between the 5 target profiles: Amazon Echo, Belkin WeMo Motion, Philips Hue Bridge, Samsung SmartCam, and TP-Link Smart Plug.
2. **Replay Engine:** Simulates real-time packet arrivals by playing historical test sessions packet-by-packet. Allows users to Pause, Step forward window-by-window, or accelerate playback up to 5x.
3. **Rolling IAT Timeline Chart (`IatTimelineChart.jsx`):** An interactive time-series line chart rendering the last 128 inter-arrival times. Hovering over any point displays the exact microsecond delay between those two packets.
4. **Live 13 Statistical Moments Table (`FeaturesTable.jsx`):** Dynamically updates every 128 packets, displaying how central tendency, quantiles, and entropy shift as device activity changes.
5. **Probability Distribution Bars (`ProbabilityBars.jsx`):** Shows the real-time posterior confidence bars across all 5 classes with vibrant theme coloring.
6. **Verdict Banner:** Emits a bold green `KNOWN` badge when confidence exceeds 84%.

4.3 Tab 2: Unknown Devices — Open-Set Benchmarking (`TabUnknown.jsx`)

Educational Purpose

To teach the concept of **Open-Set Recognition** — proving that the system does not blindly classify foreign hardware into an enrolled class.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ANOMALY THRESHOLD
SLIDER: 0.50 ■■■■■■■■■■■■● (tau=0.84) ■■■■■■■■■■■ 0.99 ■ ■ "Controls the boundary between Known and Unfamiliar
devices" ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ DEVICE
CATALOG (22 Devices from UNSW Living Lab) ■ ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ Nest DropCam ■ ■ Netatmo Weather ■ ■ Belkin Switch■ ■ ■ ■ Evaluated: 500 wins ■ ■
Evaluated: 358 wins ■ ■ Eval: 500 ■ ■ ■ ■ Rejection Rate: 100% ■ ■ Rejection Rate: 100% ■ ■ Rej Rate: 31%■ ■ ■ ■
[REJECTED - UNKNOWN] ■ ■ [REJECTED - UNKNOWN] ■ ■ [MIXED LEAK] ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■ ■ ... 15 Active Evaluated Devices | 7 Inactive / Offline
Devices ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Key UI Elements Explained

1. **The Interactive Threshold Slider ($\tau \in [0.50, 0.99]$):**

* *What it does:* Allows learners to drag the confidence cutoff in real time.
* *Learning Insight:* Dragging $\tau$ down to $0.50$ shows foreign devices leaking through as false positives. Dragging $\tau$ up to $0.95$ shows legitimate devices getting falsely rejected. $\tau = 0.84$ visually demonstrates the sweet spot where foreign hardware is rejected while known hardware is verified.

2. **The 22-Device Catalog Grid:**

- Shows every non-training device from the UNSW benchmark.
- **15 Active Devices:** Cards display the number of evaluated windows, the dominant closed-set match, average confidence, and the percentage flagged as low-confidence unknown.

* **7 Offline Devices:** Badged as *"Device was offline in capture window"*, explaining why they have 0 packets.

3. **The Firmware Sharing Lesson Card (Belkin Switch):**

- Demonstrates that `BelkinWeMoSwitch` is rejected only 30.8% of the time (69.2% falsely accepted as Belkin Motion Sensor). It teaches learners that IAT analysis fingerprints software loops; devices sharing identical firmware cannot be distinguished by timing alone.

4.4 Tab 3: Test Your Own Device — Custom Traffic Ingestion (`TabTestOwn.jsx`)

Educational Purpose

To provide an interactive **testing laboratory** where users can input custom traffic, upload real capture files, or paste raw timestamps, and immediately observe the pipeline's decision.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ INGESTION MODE: ( )
Upload .pcap/.csv ( ) Live Sniff (•) Paste ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ PASTE PACKET
TIMESTAMPS: ■ ■ [ 1479859200.000, 1479859200.009, 1479859200.411 ... ] ■ ■ ■ ■ [ Classify Pasted Timestamps ] ■ Load
Sample Echo Timestamps ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ DEVICE-LEVEL
AGGREGATE VERDICT: ■ ■ My Test Device: UNKNOWN ■ ■ Profile: Unfamiliar Device (Below Threshold) · 129 timestamps ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ WINDOW SUMMARY
TABLE: ■ ■ Win # | Verdict | Assigned | Confidence | Echo | Belkin | Hue | Cam ■ ■ #1 | UNKNOWN | Echo | 54.0% | 54%
| 16% | 22% | 8% ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
INSPECTOR: Probability Bars & 13 Statistical Moments Panel ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Key UI Elements Explained

1. **Three Input Modes:**

* *Upload Capture:* Ingests `.csv` or `.pcap` files with automatic timestamp parsing.

* *Live Network Sniffing:* Connects to local interface streams.
* *Paste Timestamps:* Direct text area accepting space-, comma-, or newline-separated epoch floats.

2. **One-Click Sample Loader:** Clicking *"Load Sample Echo Timestamps"* injects authentic 129-packet Amazon Echo traffic instantly for quick demonstrations.
3. **Multi-Window Aggregate Banner:** Displays the overall device-level decision using standardized soft probability averaging across all windows.
4. **Window Summary Table:** Lists every 128-packet window. Clicking any row opens the detailed Inspector panel below, displaying that exact window's 13 moments and probability bars.
5. **Export Buttons (`Export CSV`, `Export JSON`):** Allows researchers to export the computed features and classification decisions for external verification.

4.5 Tab 4: Spoofing Detection & Identity Verification (`SpoofCheck.jsx`)

Educational Purpose

The flagship extension of this project: demonstrating how passive timing analysis provides **Zero-Trust Identity Verification** against MAC address cloning and class impersonation attacks.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ SUB-TAB: [
Verification Studio ] [ 5x5 Matrix ] [ Adversarial Curves ]■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ IDENTITY
VERIFICATION STUDIO ■ ■ ■ ■ Physical Transmitter (Hardware Wire): [ Amazon Echo ▼ ] ■ ■ Claimed Identity (Packet MAC
Header): [ Philips Hue Bridge ▼ ] ■ ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ ■■ VERDICT:
SPOOF_SUSPECTED ■ ■ ■ ■ Physical profile matches Amazon Echo (100%), but asserted identity■ ■ ■ ■ in packet header
is Philips Hue. Rogue impersonator detected! ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ 5x5 CROSS-DEVICE
IMPERSONATION MATRIX ■ ■ Filters: [•] Total Rejection % [ ] Spoof Suspected % [ ] Bypass % ■ ■ ■ ■ Actual \ Claimed
Echo Belkin Hue Camera Plug ■ ■ Echo 12.2% FR 100% REJ 100% REJ 100% REJ 100% REJ ■ ■ Belkin 100% REJ 1.4% FR 100%
REJ 100% REJ 100% REJ ■ ■ Hue 100% REJ 100% REJ 3.3% FR 100% REJ 100% REJ ■ ■ Camera 100% REJ 100% REJ 100% REJ 1.6%
FR 100% REJ ■ ■ Plug 100% REJ 100% REJ 100% REJ 100% REJ 0.0% FR ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ADVERSARIAL STRESS
CURVES: Quantile Morphing vs Jitter Noise ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Key UI Elements Explained

1. **The Three-Identity Verification Studio:**

* Users select what the device *actually is* (Physical Transmitter) and what it *claims to be* in its network header (Claimed Identity).

- **Three Visual Security Badges:**
- `VERIFIED` (Emerald Green): Timing matches the claimed identity ($\text{Pred} = \text{Claimed}$ and $\text{Conf} \ge 0.84$).
- `SPOOF_SUSPECTED` (Vibrant Rose/Red): Physical timing matches another authorized device, but contradicts the claimed identity. **Active insider attack caught.**
- `UNKNOWN_HARDWARE` (Amber): Confidence $< 0.84$. Unauthorized foreign transmitter.

2. **The 5x5 Cross-Device Impersonation Heatmap:**

- Visualizes all 20 cross-device attack pairs.
- **Interactive Metric Filters:**

* *Total Rejection %:* Shows 100% across all 20 attack pairs.
* *Spoof Suspected %:* Shows the percentage caught via direct profile identification (96.29% mean).
* *Bypass Rate %:* Shows **0.00%** across the entire matrix (proving zero successful impersonations).

- Diagonal cells display legitimate False Rejection rates ($1.4\%$–$12.2\%$).

3. **Adversarial Timing Disturbance Curves:**

- Plots attack strength $\alpha \in [0.0, 1.0]$ against detection rate.
- Shows that **timing jitter noise is 100% ineffective** against the model, while sophisticated **quantile morphing** degrades detection when distribution imitation exceeds 80%.

4. **Multi-Window Soft Probability Smoothing Toggle:**

- Demonstrates how rolling 5-window soft probability averaging ($\frac{1}{5}\sum \mathbf{P}_k$) boosts legitimate verification to **98.10%** and suppresses transient false alarms.

---

5. Master Dictionary of UI Parameters & Metrics

When demonstrating or defending the dashboard, use this exact reference table:

Parameter on Screen	Mathematical Formula	Physical & Operational Meaning
`Mean`	$\mu = \frac{1}{W}\sum z_i$	Average log-transformed IAT. Measures baseline packet transmission pacing.

`Std Dev`	$\sigma = \sqrt{\frac{1}{W}\sum(z_i - \mu)^2}$	Jitter/fluctuation of packet arrivals around the mean.
`Min IAT`	$\min(\mathbf{z})$	The fastest back-to-back packet transmission speed (governed by network hardware).
`Max IAT`	$\max(\mathbf{z})$	The longest idle interval or sleep cycle within the 128-packet window.
`Median`	50th Percentile Q_2	The robust center of traffic pacing, unaffected by occasional sleep outliers.
`P25`	25th Percentile Q_1	Pacing of rapid conversational bursts between endpoint and cloud.
`P75`	75th Percentile Q_3	Pacing of typical inter-packet pauses. **(#1 most important feature in our model).**
`P95` **&** `P99`	95th & 99th Percentiles	Tail boundaries capturing periodic keep-alives and beacon timeouts.
`Skewness`	$\frac{1}{W}\sum((z_i - \mu)/\sigma)^3$	Asymmetry. Positive skew indicates bursts near zero with long sleep tails.
`Kurtosis`	$\frac{1}{W}\sum((z_i - \mu)/\sigma)^4 - 3$	Peakedness / tail heaviness. Measures the presence of extreme sporadic spikes.
`Signal Energy`	$\sum z_i^2$	Total signal power of the timing window.
`Entropy`	$-\sum p_i \log_2(p_i)$	Shannon spectral entropy. Measures the randomness/unpredictability of timing.
`Confidence`	$\max_c P(y = c \mid \mathbf{x})$	The percentage of the 100 Random Forest trees voting for the winning class.
`Threshold` **(τ)**	$\tau = 0.8400$	The minimum confidence required to accept a device as known. Calibrated at 95th %ile of validation error.
`Window Size` **(W)**	$W = 128$	The fixed batch size of 128 consecutive IATs (spanning 129 physical packets).

---

6. Backend REST API Architecture & Route Catalog

The FastAPI server exposes structured endpoints documented automatically via Swagger at `http://127.0.0.1:8765/docs`:

API Endpoints Catalog

1. `GET /health`

* *Returns:* `{"status": "ok", "service": "iot-fingerprinting-backend", "port": 8765}`.
* *Purpose:* Health probe used by the frontend to confirm backend connectivity.

2. `GET /model/info`

* *Returns:* Model type, number of trees (100), enrolled classes, window size (128), and calibrated threshold (0.84).

3. `GET /evaluation`

* *Returns:* Complete precomputed evaluation metrics for the 5 known devices and 22 benchmark catalog devices.

4. `GET /stream/{device_name}`

* *Parameters:* `device_name` (e.g. `AmazonEcho`).
* *Returns:* Pre-windowed historical packet sessions for Tab 1 replay streaming.

5. `POST /predict`

* *Body:* JSON containing an array of 13 unscaled feature values.
* *Returns:* Predicted profile, class posterior probabilities, and verdict (`KNOWN` vs `UNKNOWN`).

6. `POST /predict/csv`

* *Body:* Raw CSV or PCAP file upload.
* *Returns:* Processed windows, per-window features, and device-level aggregate verdict.

7. `GET /spoof/results`

* *Returns:* Offline benchmark data for Tab 4 (impersonation matrices, mimicry curves, group verified rates).

8. `POST /spoof/verify`

* *Body:*
`json

{

"timestamps": [1479859200.12, 1479859200.18, ...],

"claimed_identity": "PhilipsHue",

"threshold": 0.84,

"n_group": 5

}
`

* *Validations Enforced:*
  - Rejects invalid `claimed_identity` not in `CLASSES` with HTTP 400.
  - Rejects non-finite timestamps (`NaN`, $\pm\infty$) with HTTP 400.
  - Requires at least 2 timestamps to compute IATs.
* *Returns:* 3-state verdict (`VERIFIED`, `SPOOF_SUSPECTED`, `UNKNOWN_HARDWARE`), confidence, window verdicts, and soft probability averages.

---

7. Step-by-Step 5-Minute Demonstration Script

Use this exact script when presenting or demonstrating the live website:

Minute 1: The Core Problem & Tab 1 (Known Devices)

* *Action:* Open `http://localhost:5173/` in your browser. Navigate to **Tab 1 ("Known Devices")**.
* *What to Say:*
> *"Modern smart home devices encrypt their communications using TLS, meaning firewalls cannot inspect packet payloads. However, every device has a unique physical hardware signature in the timing between its packets. In Tab 1, we can see real-time replays of our 5 enrolled devices. Notice how the Amazon Echo's rolling timeline shows frequent microsecond bursts, whereas the TP-Link plug sleeps for minutes at a time. The Random Forest extracts 13 statistical moments and classifies known devices with 99.34% accuracy."*

Minute 2: Tab 2 (Open-Set Recognition & Thresholding)

* *Action:* Click on **Tab 2 ("Unknown Devices")**.
* *What to Say:*
> *"In a real enterprise network, unknown devices will connect. A standard closed-world classifier dangerously forces unknown hardware into an enrolled class. In Tab 2, we show how our calibrated threshold ($\tau = 0.84$) rejects foreign devices. If I drag the threshold slider down to 0.50, foreign hardware leaks through. At our calibrated 0.84 boundary, 15 real foreign IoT devices achieve a 95.66% rejection rate. Cameras and smart sensors reject at 100%, while devices sharing identical vendor firmware, like the Belkin Switch, demonstrate why timing analysis fingerprints firmware loops rather than brand logos."*

Minute 3: Tab 3 (Live Timestamp Ingestion)

* *Action:* Click on **Tab 3 ("Test Your Own Device")**. Paste the **Nest DropCam** sample timestamps (129 timestamps). Click **"Classify Pasted Timestamps"**.
* *What to Say:*
> *"Here, we test real unclassified packet timestamps. We paste 129 timestamps from a Nest DropCam. The pipeline segments the window, extracts the 13 moments, and evaluates confidence. Notice the result: it flags as UNKNOWN / Unfamiliar Device. Even though the closest mathematical profile is Amazon Echo, its confidence is only 54% — well below our 84% cutoff. The system safely refuses to make a false identification."*

Minute 4: Tab 4 (Spoofing & Identity Verification Studio)

* *Action:* Click on **Tab 4 ("Spoofing Detection")**. Select **Physical Transmitter: Amazon Echo** and **Claimed Identity: Philips Hue Bridge**.
* *What to Say:*
> *"Now we demonstrate active anti-spoofing. Suppose an attacker clones the MAC address of an authorized Philips Hue lighting controller onto an Amazon Echo or laptop. The network header claims it is a Philips Hue, but our physical timing engine identifies the transmitter as Amazon Echo with 100% confidence. Notice the verdict immediately flips to a bold red SPOOF_SUSPECTED badge! We have caught an insider impersonation attack without decrypting a single payload."*

Minute 5: 5x5 Impersonation Matrix & Adversarial Curves

* *Action:* Scroll down in Tab 4 to the **5x5 Impersonation Matrix**. Toggle the **Bypass Rate %** filter. Point to the **Adversarial Disturbance Curves**.
* *What to Say:*
> *"Finally, we evaluated all 20 cross-device impersonation attack combinations across our entire test split. As shown in this 5x5 matrix, our total rejection rate is 100.0%, and our bypass rate is exactly 0.00% — zero attacks bypassed the defense. Furthermore, our stress-testing curves show that random timing jitter noise is 100% ineffective at fooling the model, proving that physical timing analysis provides robust zero-trust endpoint authentication."*

---

8. Command Reference & Maintenance Guide

How to Start Both Servers

```
# 1. Start FastAPI Backend (Port 8765): cd C:\Users\Murali\Desktop\cn-proj\two\iot-fingerprinting python -m uvicorn
iot_fingerprint_gui.backend.main:app --host 127.0.0.1 --port 8765 # 2. Start Vite React Frontend (Port 5173): cd
C:\Users\Murali\Desktop\cn-proj\two\iot-fingerprinting\iot_fingerprint_gui\frontend npm run dev
```

Common Troubleshooting Checklist

1. Frontend says "Connecting to Backend...":

- Verify FastAPI is running in terminal on `http://127.0.0.1:8765`.
- Open `http://127.0.0.1:8765/health` in your browser. It should return `{"status": "ok"}`.

2. "Need at least 129 timestamps" error in Tab 3:

- A single 128-packet IAT window requires $N = 129$ timestamps because $\text{IAT}_i = t_i - t_{i-1}$ produces $N - 1$ intervals. Ensure at least 129 timestamps are pasted.

3. HTTP 400 "Invalid timestamp input":

- One of your timestamps contains non-numeric text, `NaN`, or infinity. Sanitize your input data before pasting.

Report compilation complete. Formatted for comprehensive educational understanding and live software presentation.

Visual Evidence & Dashboard Figures

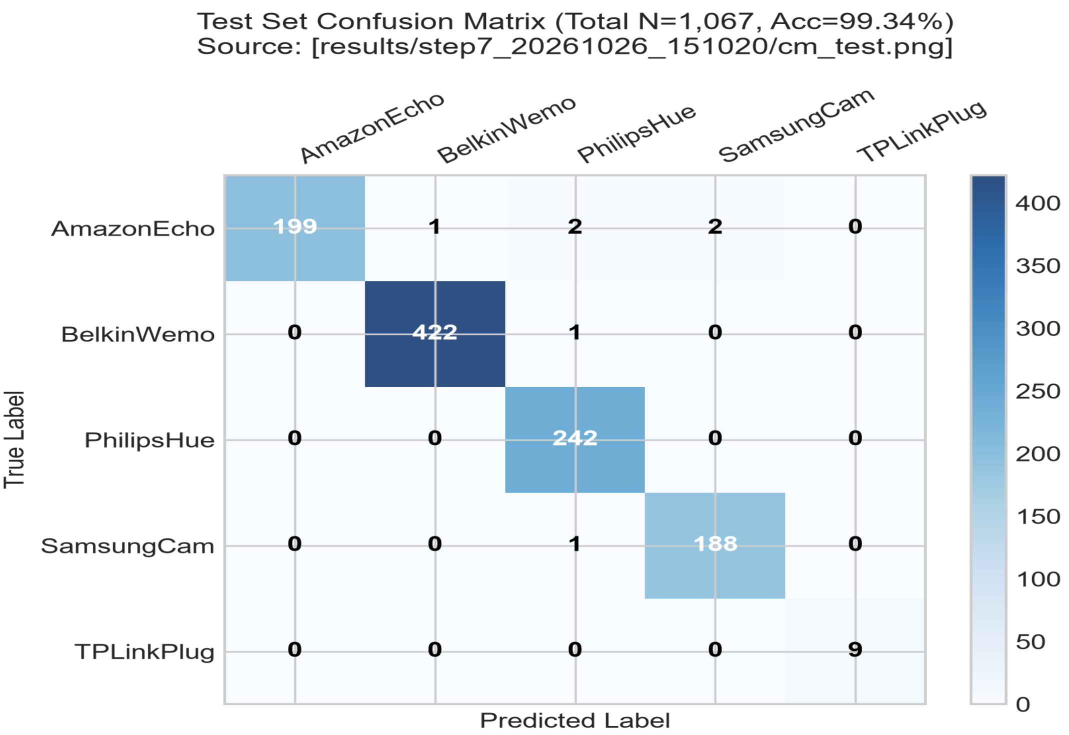

Figure 1: Tab 1 Baseline Performance — Closed-Set Test Confusion Matrix (99.34% Accuracy)

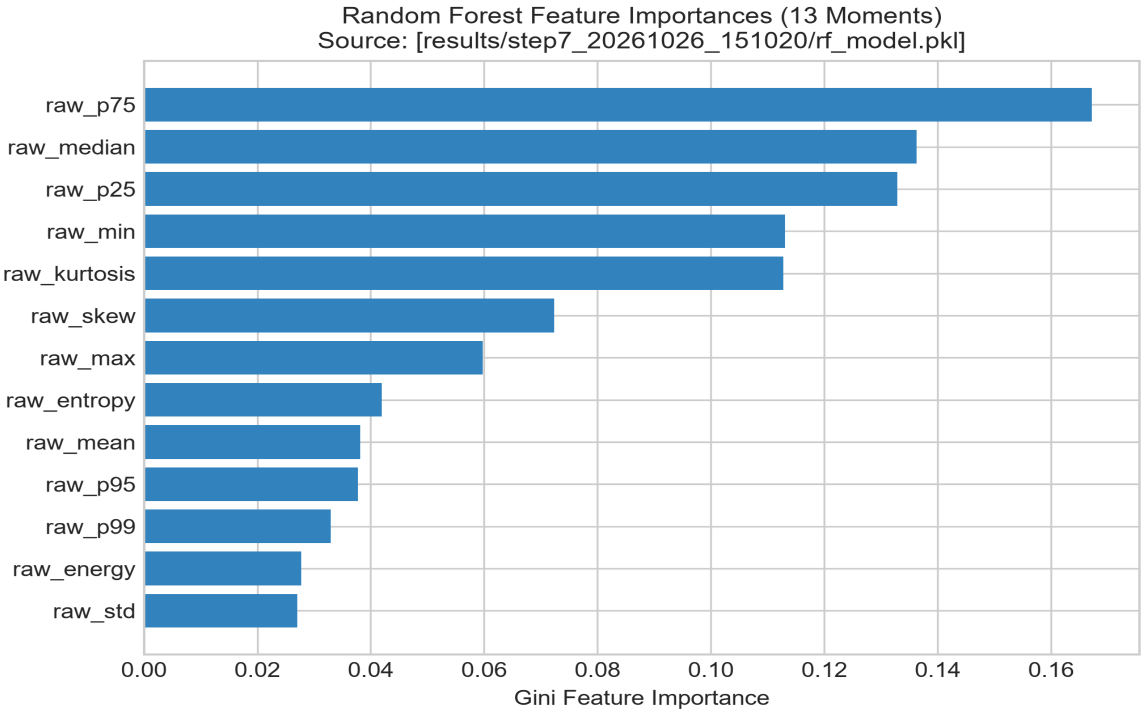

Figure 2: The 13 Moments in Tab 1 — Random Forest Gini Feature Importances

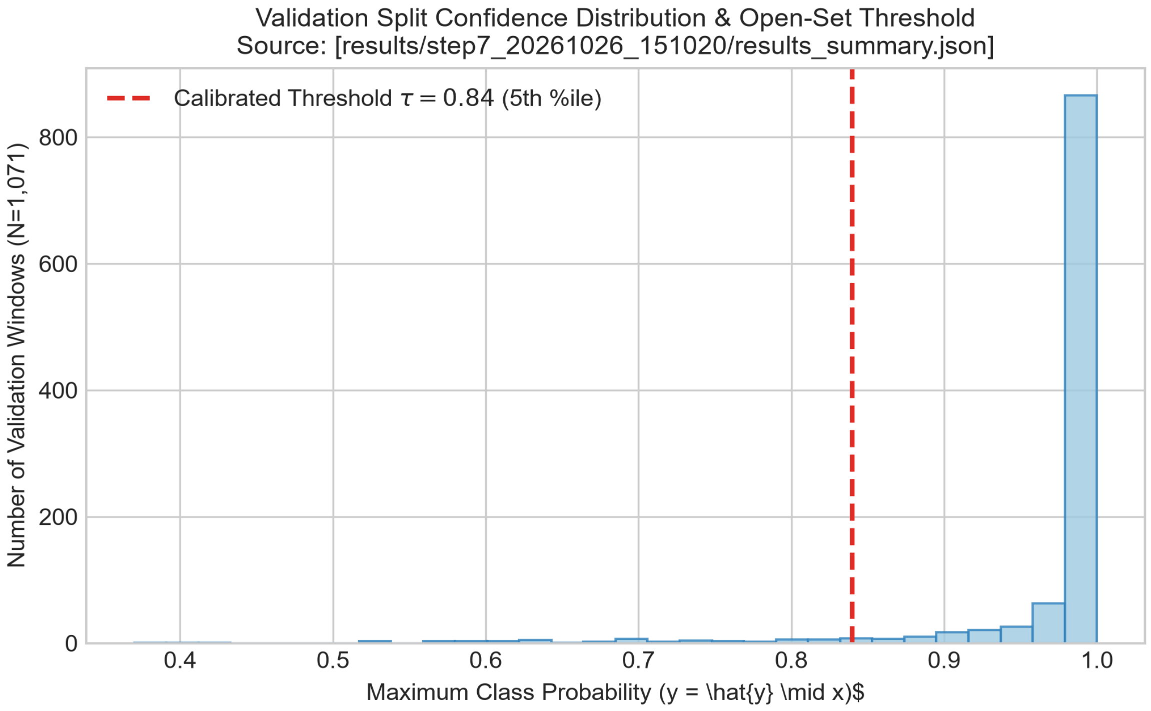

Figure 3: Tab 2 The Threshold Slider — Validation Confidence & tau=0.84 Boundary

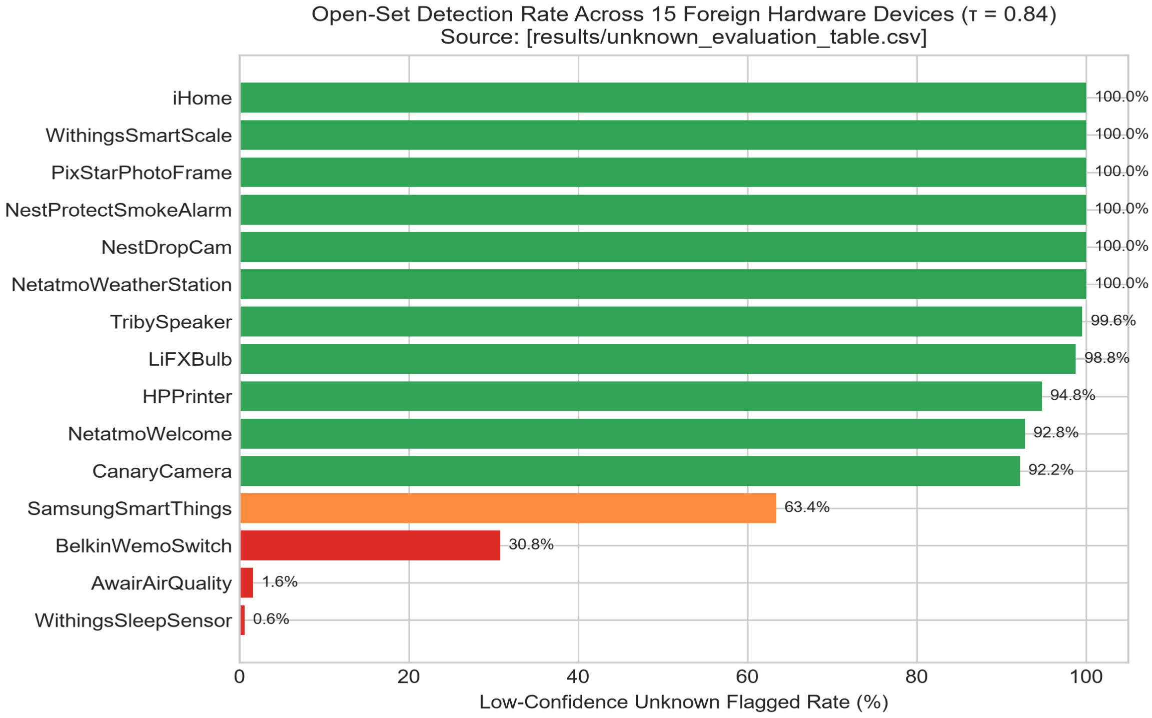

Figure 4: Tab 2 Device Catalog — Open-Set Detection Across 15 Foreign Hardware Devices

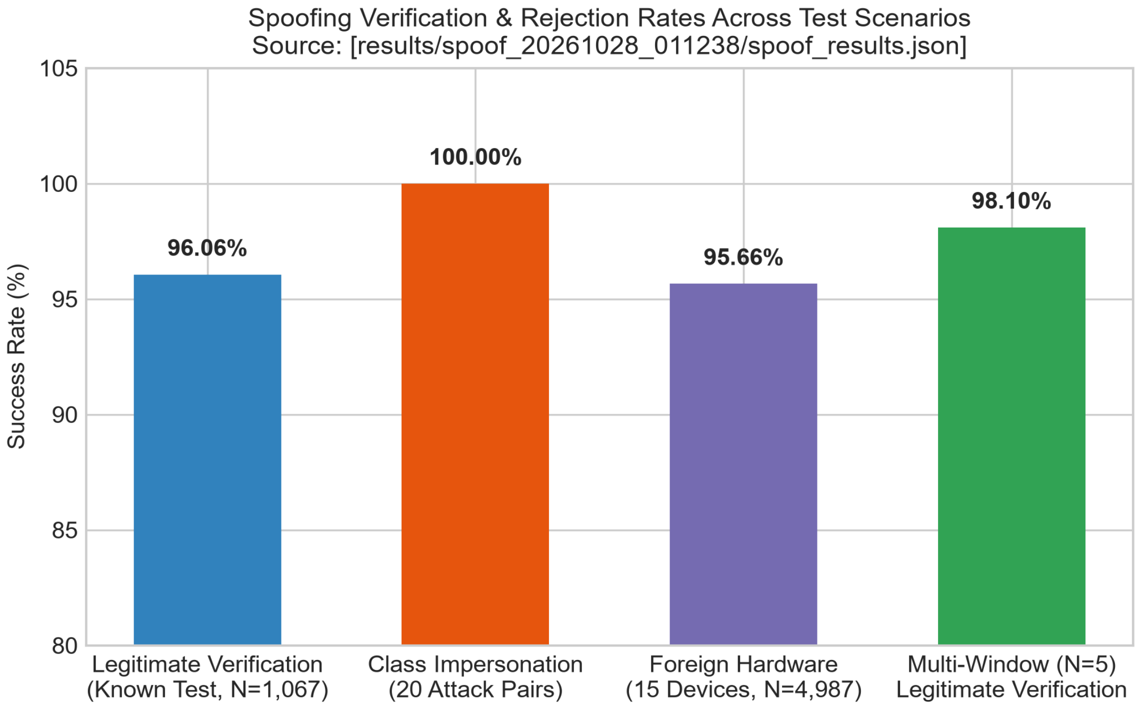

Figure 5: Tab 4 Anti-Spoofing — Rejection Rates Across 4 Security Scenarios

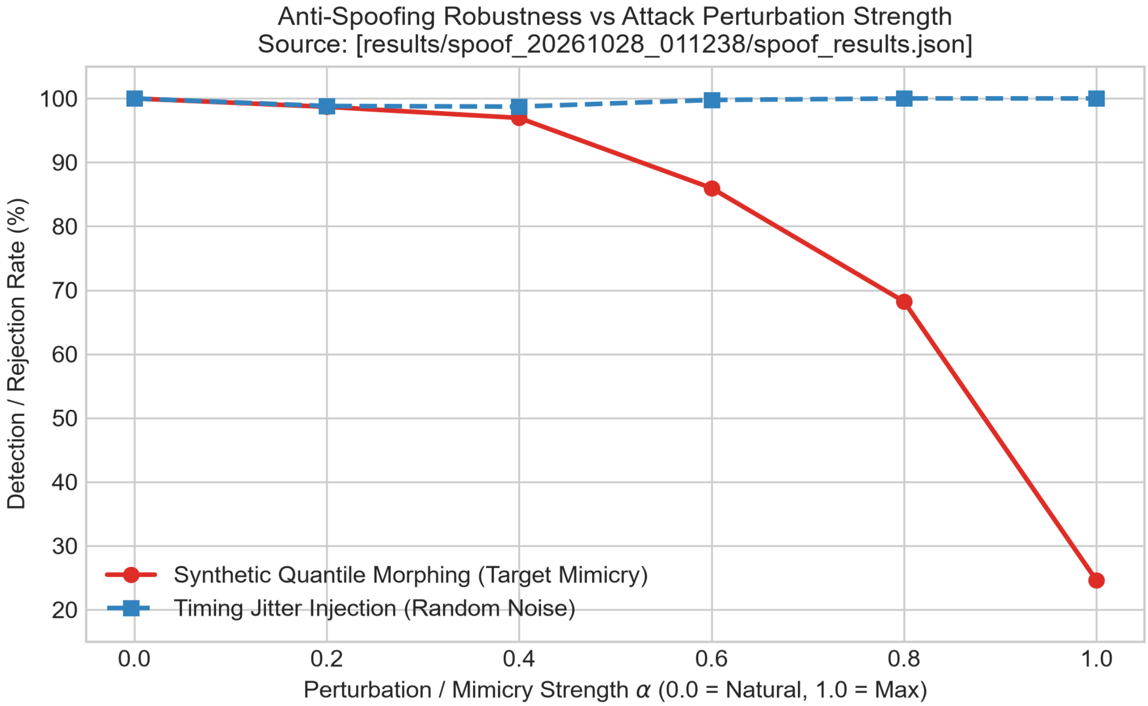

Figure 6: Tab 4 Stress Testing — Jitter Robustness vs Quantile Morphing Degradation